

Constructing Abelian Square-Free Words and Testing Morphisms with Mathematica

Ville Mattila
Ville.Mattila@ccc.fi
<http://south.rotol.ramk.fi>

Abstract

My final year project [2] covers two different approaches on combinatorics on words:

- i. Dr Veikko Keränen came up with the idea of creating a-2-free words using polynomial inequalities. It's a useful method when constructing and studying a-2-free structures in words.
- ii. Arturo Carpi's method for testing a-2-freeness of morphisms.

The project was commissioned by Keränen who has actively done research on a-2-free words and structures over a four letter alphabet. Review Keränen's paper (*New Abelian Square-Free DTOL-Languages over 4 Letters*) in this *IAS 2002* proceedings for an extensive explanation of the terminology used in this paper.

The *Mathematica* packages including the functions mentioned in this paper can be found from Keränen's web server <http://south.rotol.ramk.fi> in the near future.

This paper covers a brief introduction to these two methods and their *Mathematica*-implementations.

1 Polynomial Inequalities

Recall that a word w is a-2-free, if it does not contain such consecutive (non-empty) subwords u and v for which $\psi(u) - \psi(v) = \overline{0}$.

Why to use polynomial inequalities?

We can replace some parts of the word with letter variables. Then we construct all possible subwords u and v and form the differences of their Parikh-vectors. After that we form the product of the differences. All the values of variables for which the product is unequal to zero form an a-2-free word.

Example

Let $w = "aXab"$. Let's solve the possible values for " X ".

We define a function that constructs the subword differences for a given word:

```
vmmSubwordDiffs[word_String] := Module[{myDiffs = {}},

(*
  Three different pointers are used.
  e points to the end of the subword v.
  m points to the beginning of the subword v.
  Thus m-1 is the end of subword u.
  s is the start pointer of subword u.
*)

e = 2;

While[e <= StringLength[word],
  m = e;
  b = m - 1;
  While[b > 0,
    v = StringTake[word, {m, e}];
    u = StringTake[word, {b, m - 1}];
    pv = Apply[Plus, Characters[v]]; (* the Parikh-vector *)
    pu = Apply[Plus, Characters[u]];
    AppendTo[myDiffs, (pv - pu)];
    m = m - 1;
    b = b - 2;
  ];
  e = e + 1;
];

Return[myDiffs]

];
```

As you have noticed, I used procedural structure in the function that creates the subword differences. Generally I find Mathematica's pattern matching to be quite a versatile feature but in this case I could not figure out a pattern matching function that would have been faster than the procedural one. In many places the use of lists was really helpful. When we use lists we can also use set operations such as `Outer` or `Union`. We can also save several lines of code when using lists instead of strings.

Now we can use the self-made function to get the subword differences:

```
myWord = "aXab";

myDiffs = vmmSubwordDiffs[myWord]

{-a + X, a - X, -a + b, b - X}
```

So, here we have a nice list of the differences. Notice, that Mathematica has simplified the last difference $(a + b) - (a + X)$ to the form $(b - X)$.

Now we can construct the product:

```
myProduct = Times @@ myDiffs

(-a + b) (a - X) (b - X) (-a + X)
```

Before we can solve the product, we must define the alphabet to be used, ie., the possible values for the letter variable.

```
myAlphabet = Characters["abcd"];
```

Now we are ready to solve the product:

```
myResult = {};
If[ (myProduct /. {"X" -> #}) != 0,
  AppendTo[myResult, "X" -> #];
] & /@ myAlphabet;
myResult

{X -> c, X -> d}
```

And here's the solution! We can now apply this rule to the original word:

```
StringReplace[myWord, #] & /@ myResult

{acab, adab}
```

About the functions

Basically the functions in my package work in the way explained in the example above. The package is still in developmental level as the construction of the words is not very effective. Converting the strings to lists (or letters) is taking too much time. Also the use of the empty set would be very useful. Marianna Mölläri, an engineering student of Rovaniemi Polytechnic, has already started her final year project which is also commissioned by Keränen. One of her aims is to develop the functions in my polynomial inequalities package to a more simple and effective form.

More examples

Here are some examples of how the functions in my package can be used.

```
<< vmmPolyTests`vmmPolyTests`

vmmMakeA2FreeWord["aXaYaZaTaKaLaMa"]

{abacabadabacaba, acabacadabacaba, abacadacabacada,
 abadabacabadaba, adabadacabadaba, abadacabadadaca,
 abacabadacabaca, acabacadacabaca, acabadabacabada, acadabadacadaba,
 acadacabadadaca, adacadabadadaca, adabacabadabaca, abadabacadabada,
 adabadacadabada, adacabadacadaba, acadacabadacada, adacadabadacada}

vmmMakeA2FreeWord["aXaYaZaTaKaLaMaNa"]

{ }
```

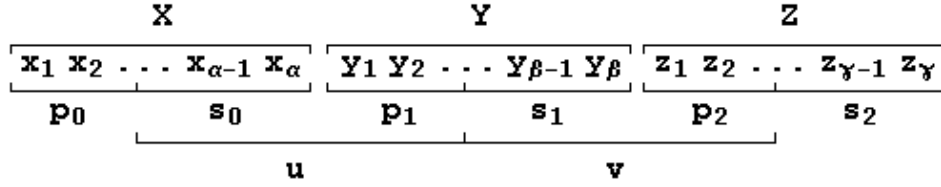
As the above example shows, it is possible to create an a -2-free structure of the length 15 that has an a as the every other letter and some other letter between all the occurrences of a . But when we increase the length to 17 letters, we can see that no such structure exists.

2 Carpi's Algorithm

Carpi's characterisation of a -2-free morphisms can be found in [1] or in Keränen's paper in this proceedings. The principle of the algorithm implemented in [2] is explained in this paper.

The relation between prefixes and subwords u and v

The following is an illustration of the relation between prefixes of the image words $\{X, Y, Z\}$ of a morphism (for letters), and subwords u and v , for which the word uv may form an abelian square. For simplicity, we omit here all the blocks in $g(\Sigma^*)$ between the words X, Y, Z , ie., in the border lines of $X|Y|Z$.



Indeed, from this pictorial representation one derives

$$\psi(v) - \psi(u) = \psi(p_2) - 2\psi(p_1) + \psi(p_0) + \psi(Y) - \psi(X) ;$$

see Section 6 (5) of Keränen's paper in this proceedings.

Principle of the algorithm:

In what follows, we write simply XYZ instead of the triplet $\{X, Y, Z\}$.

Consider, for example, the case of $XYZ = ACA$.

Form all differences of subwords u and v .

If all $\psi(v) - \psi(u) \neq \bar{0}$, we can add the same letter (image word) on both sides of the occurrence of the middle letter, as long as the underlying word remains a-2-free, and all $\psi(v) - \psi(u)$ still remain $\neq \bar{0}$. Using this principle we can form the word $ABCBA$ from the word ACA , but not the word $ACCCA$, since the underlying word of it is not a-2-free. However, $ACCCA$ can be extended, in a balanced way, into $ABCD C BCD A$ the underlying word of which is a-2-free.

As we go on, the balance, say in $ABACABA$, is still all right, but not e.g. in $ACCCACA$, which actually is an example of a non problematic case, since the underlying word will always contain an abelian square when balanced words are added to both sides of the middle occurrence of the word C .

Note that cases like $ABACA$ and $ACABA$ will be tested in the case of $XYZ = AAA$.

Thus we have to check the cases in which the subwords u and v are such that $\bar{v} - \bar{u} = \alpha \bar{A} + \beta \bar{B} + \gamma \bar{C} + \delta \bar{D}$ ($\{\alpha, \beta, \gamma, \delta\} \in \mathbb{Z}$).

The test has to be repeated on every three-letter word ($AAA, AAB, AAC, \dots, DDC, DDD$). If the imagewords are cyclic permutations of each other, we need only to test words from AAA to ADD .

An example of a problematic structure found in the test

Assume that

$$\psi(u) - \psi(v) = \psi(B) - \psi(A)$$

and

$$C = puvs.$$

Consider the a-2-free word $cacbc$. In this case, the image word of $cacbc$ is never a-2-free:

$$\begin{aligned} g(cacbc) &= CACBC = puvs A puvs B puvs \\ &puv (sApu) (vsBp) uvs \end{aligned}$$

Thus

$$\begin{aligned} \psi(sApu) - \psi(vsBp) &= \psi(Au) - \psi(vB) \\ &= \psi(A) + \psi(u) - (\psi(v) + \psi(B)) \\ &= \psi(u) - \psi(v) - (\psi(B) - \psi(A)) \\ &= \{0, 0, 0, 0\} \end{aligned}$$

Consequently, $(sApu)(vsBp)$ is an abelian square.

This problematic structure is being found when testing the word CCC .

The test is simple, if we use matrices and vectors:

- Prefixes are used as Parikh-vectors, i.e. $\{1, 0, 2, 3\}$ etc.
- Parikh-vectors of the image words form a matrix

See the following for how the matrices are used in detail:

Let's form the matrix

$$\begin{aligned} \psi[A] &= \{\#_a[A], \#_b[A], \#_c[A], \#_d[A]\}; \\ \psi[B] &= \{\#_a[B], \#_b[B], \#_c[B], \#_d[B]\}; \\ \psi[C] &= \{\#_a[C], \#_b[C], \#_c[C], \#_d[C]\}; \\ \psi[D] &= \{\#_a[D], \#_b[D], \#_c[D], \#_d[D]\}; \end{aligned}$$

$$\text{myMatrix} = \{\psi[A], \psi[B], \psi[C], \psi[D]\} // \text{TraditionalForm}$$

$$\begin{pmatrix} \#_a(A) & \#_b(A) & \#_c(A) & \#_d(A) \\ \#_a(B) & \#_b(B) & \#_c(B) & \#_d(B) \\ \#_a(C) & \#_b(C) & \#_c(C) & \#_d(C) \\ \#_a(D) & \#_b(D) & \#_c(D) & \#_d(D) \end{pmatrix}$$

Let $\psi(v) - \psi(u) = \text{diff}$

$$\mathbf{diff} = \{\Delta_a, \Delta_b, \Delta_c, \Delta_d\};$$

Problematic cases are formed as follows:

$$\begin{aligned} \mathbf{x} \#_a [\mathbf{A}] + \mathbf{y} \#_a [\mathbf{B}] + \mathbf{z} \#_a [\mathbf{C}] + \mathbf{t} \#_a [\mathbf{D}] &== \Delta_a; \\ \mathbf{x} \#_b [\mathbf{A}] + \mathbf{y} \#_b [\mathbf{B}] + \mathbf{z} \#_b [\mathbf{C}] + \mathbf{t} \#_b [\mathbf{D}] &== \Delta_b; \\ \mathbf{x} \#_c [\mathbf{A}] + \mathbf{y} \#_c [\mathbf{B}] + \mathbf{z} \#_c [\mathbf{C}] + \mathbf{t} \#_c [\mathbf{D}] &== \Delta_c; \\ \mathbf{x} \#_d [\mathbf{A}] + \mathbf{y} \#_d [\mathbf{B}] + \mathbf{z} \#_d [\mathbf{C}] + \mathbf{t} \#_d [\mathbf{D}] &== \Delta_d; \end{aligned}$$

The same in matrix form:

$$\begin{pmatrix} \#_a(\mathbf{A}) & \#_a(\mathbf{B}) & \#_a(\mathbf{C}) & \#_a(\mathbf{D}) \\ \#_b(\mathbf{A}) & \#_b(\mathbf{B}) & \#_b(\mathbf{C}) & \#_b(\mathbf{D}) \\ \#_c(\mathbf{A}) & \#_c(\mathbf{B}) & \#_c(\mathbf{C}) & \#_c(\mathbf{D}) \\ \#_d(\mathbf{A}) & \#_d(\mathbf{B}) & \#_d(\mathbf{C}) & \#_d(\mathbf{D}) \end{pmatrix} \cdot \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{z} \\ \mathbf{t} \end{pmatrix} == \begin{pmatrix} \Delta_a \\ \Delta_b \\ \Delta_c \\ \Delta_d \end{pmatrix};$$

Let's solve the vector $\{x, y, z, t\}$

$$\begin{pmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{z} \\ \mathbf{t} \end{pmatrix} == \text{Inverse} \left[\begin{pmatrix} \#_a(\mathbf{A}) & \#_a(\mathbf{B}) & \#_a(\mathbf{C}) & \#_a(\mathbf{D}) \\ \#_b(\mathbf{A}) & \#_b(\mathbf{B}) & \#_b(\mathbf{C}) & \#_b(\mathbf{D}) \\ \#_c(\mathbf{A}) & \#_c(\mathbf{B}) & \#_c(\mathbf{C}) & \#_c(\mathbf{D}) \\ \#_d(\mathbf{A}) & \#_d(\mathbf{B}) & \#_d(\mathbf{C}) & \#_d(\mathbf{D}) \end{pmatrix} \right] \cdot \begin{pmatrix} \Delta_a \\ \Delta_b \\ \Delta_c \\ \Delta_d \end{pmatrix};$$

Denote

$$\mathbf{invTranspMatrix} = \text{Inverse} \left[\begin{pmatrix} \#_a(\mathbf{A}) & \#_a(\mathbf{B}) & \#_a(\mathbf{C}) & \#_a(\mathbf{D}) \\ \#_b(\mathbf{A}) & \#_b(\mathbf{B}) & \#_b(\mathbf{C}) & \#_b(\mathbf{D}) \\ \#_c(\mathbf{A}) & \#_c(\mathbf{B}) & \#_c(\mathbf{C}) & \#_c(\mathbf{D}) \\ \#_d(\mathbf{A}) & \#_d(\mathbf{B}) & \#_d(\mathbf{C}) & \#_d(\mathbf{D}) \end{pmatrix} \right];$$

In other words:

$$\begin{pmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{z} \\ \mathbf{t} \end{pmatrix} == \mathbf{invTranspMatrix} \cdot \begin{pmatrix} \Delta_a \\ \Delta_b \\ \Delta_c \\ \Delta_d \end{pmatrix};$$

Yet another way: $(AB)^T = B^T A^T$

$$\{\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{t}\} == \{\Delta_a, \Delta_b, \Delta_c, \Delta_d\} . \text{Transpose}[\mathbf{invTranspMatrix}];$$

$(A^T)^{-1} = (A^{-1})^T$. Thus $((A^T)^{-1})^T = ((A^T)^T)^{-1} = A^{-1}$ and thus

$$\{\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{t}\} == \{\Delta_a, \Delta_b, \Delta_c, \Delta_d\} . \text{Inverse}[\mathbf{myMatrix}];$$

If x, y, z and t are integers, then we have a problematic structure unless the difference vector is an element of the so called Delta set (the Delta set is formed of the vectors of the image words under test. The structures in the Delta set would create an abelian square already in the underlying word itself and not only in the image word).

How the code was optimized

- **The first version** handled the subwords u and v as strings, and so the Parikh-vectors had to be recalculated again and again for each subword.

- * Procedural structure was heavy and slow

- * Comparison time: 30 minutes.

- **The second version** (and so far the latest version) uses prefixes

- * The test starts with a routine that appends the Parikh-vectors of prefixes to lists

- * Pattern matching can be used

- * Handling the lists is easy and fast (Outer-function)

- * Requires memory (RAM), but is fast

- * Comparison time: ~5 minutes.

Acknowledgements

I want to thank Veikko Keränen for his patient and disinterested help throughout the project. It has been very rewarding for me to work with a mathematician like him.

I wish good luck to Marianna Mölläri and Lauri Autio from Rovaniemi Polytechnic as they are starting their projects with Keränen. I wish that also in the future students in RP will participate in Keränen's research.

References

- [1] A. Carpi. On abelian power-free morphisms. Int. J. Algebra Comput., 3:151-167. World Sci. Publ. Company, 1993
- [2] V. Mattila. Creating Permutation Repetition-Free Words and Testing Permutation Repetition-Freeness of Morphisms (in Finnish). B.Sc. Thesis. Rovaniemi Polytechnic, 2002.

More of this on the internet: <http://south.rotol.ramk.fi>